

Final Report: Bike Sharing Demand

Angelina Bautista & Molly Scally

1. Executive Summary

This project is focused on solving the problem that bike-sharing systems can have – allocating resources improperly, specifically ensuring we lower costs put towards improper bike redistribution. Investigating this problem is imperative to bike-sharing systems and their operators. Finding accurate demand forecasts will help the systems ensure inventory can cover bike distribution in high-demand locations at peak hours, while avoiding unnecessary costs and losses that can come from over/under supply. Having better predictions leads to efficient operations and in turn, improved customer satisfaction.

To solve this problem, we used historical bike rental data that includes environmental and temporal data. Our goal was to predict the total number of rentals at any given hour.

After testing multiple approaches (tuned vs default), our best-performing model was a tuned Random Forest model, which achieved a strong performance with an error of approximately 44 rentals per hour (Test RMSE = 44.25). This means that, on average, the model predictions are within about 44 rentals of the true value. Given that hourly demand can reach close to 1,000 rentals at peak times, this is a strong level of accuracy and indicates the model is highly reliable for forecasting.

The most actionable finding we can present is that bike-share operators should redistribute bikes before peak commuting hours, since demand during those hours is predictably the highest and driven primarily by commuting behavior. Instead of waiting for bike stations to run empty, bike-sharing operators should proactively move bikes into high-demand pick-up areas just before peak commute time, when shortages are most likely to occur.

2. Problem & Data

The objective of the project is to develop a predictive model that estimates the total number of bike rentals (`cnt`) at any given hour in a bike-sharing system. Capital Bikeshare – a system that services Washington D.C. and surrounding areas – is the system analyzed and is the client for whom the predictive model is being developed. The prediction task is a regression task, meaning the model will predict a continuous number representing the demand for bike rentals.

Predictions are made for future time periods (hourly observations) based on provided temporal, seasonal, and weather-related features. Predictions made by the model are intended to be used by bike-sharing operators to optimize inventory distribution and system availability.

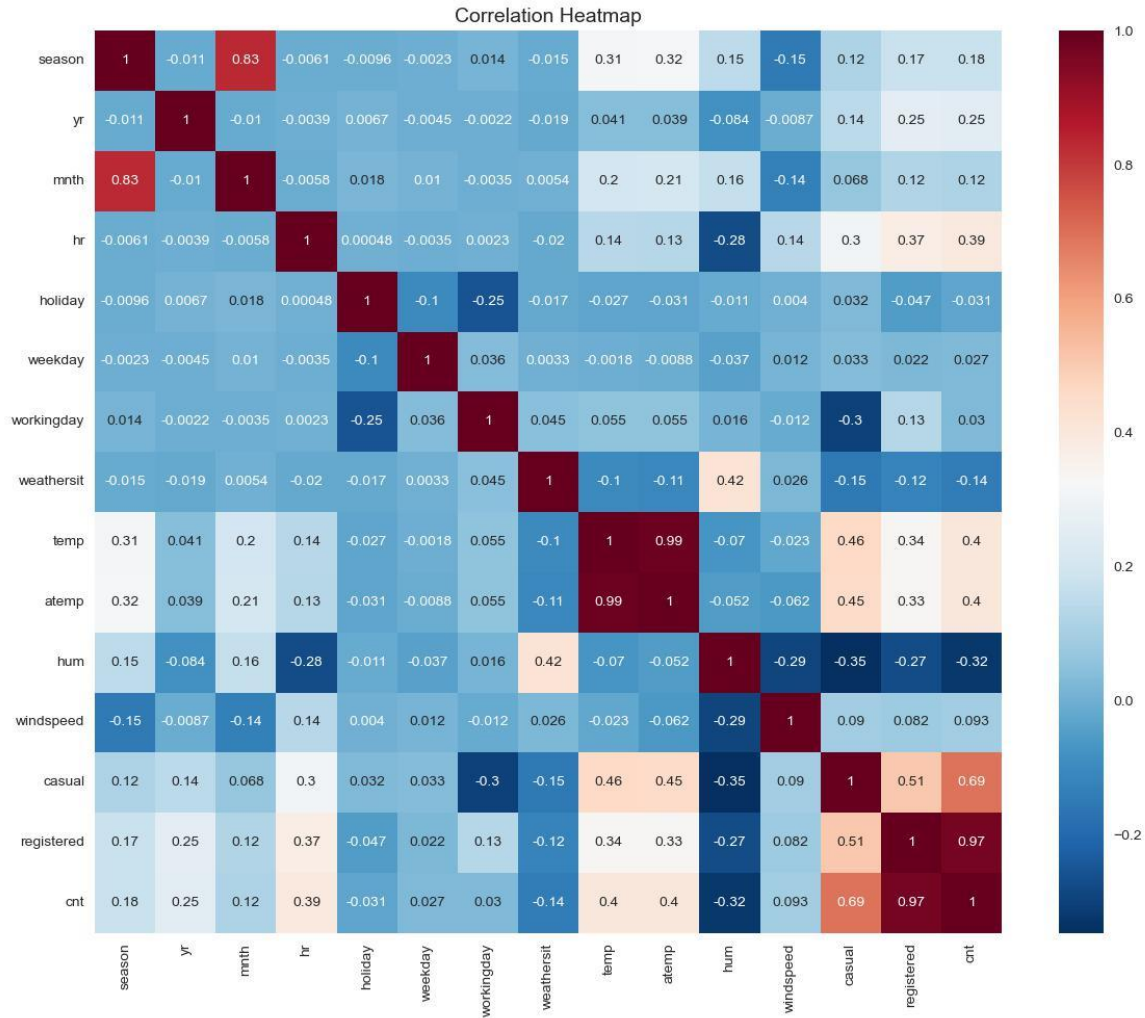
Errors made by the predictive model have practical consequences. Overestimating demand can result in unnecessary redistribution of bikes and increased operational costs – which is exactly what the task description warns about. On the other hand, underestimating demand can have a domino effect: bike shortages in high-demand pick-up locations can reduce customer satisfaction, ultimately losing revenue for the company.

The dataset is sourced from Kaggle’s [Bike Sharing Dataset](#), containing historical bike rental data from the Capital Bikeshare system in Washington D.C. The dataset spans the period 2011-2012, making it a time-series dataset with hourly granularity. It includes 17,379 observations (rows) and 17 variables (columns), where each observation represents one hourly record of bike usage.

The target variable for the dataset is `cnt`, representing the total number of bike rentals. The range for the target typically varies from one to several hundred rentals per hour, depending on environmental factors and conditions. Based on the descriptive statistics, the target has a minimum value of 1 and a maximum value of 977, indicating the spread of observed values across the full data distribution. Just using the raw data, the distribution of hourly bike rentals was right-skewed, with many low-to-moderate demand periods and fewer high-demand peaks. This distribution reflects real-world usage patterns, where the demand for bike rentals fluctuates significantly based on the time of day, weather, and seasonality.

To ensure valid model performance, variables `casual` and `registered` were dropped due to the risk of data leakage. These variables are captured in the target variable `cnt` because it directly sums into the total number of bike rentals. Including these variables would allow the model to reconstruct the target, resulting in an artificially high performance without any meaningful learning.

Figure 1. Correlation Heatmap of Numerical Features

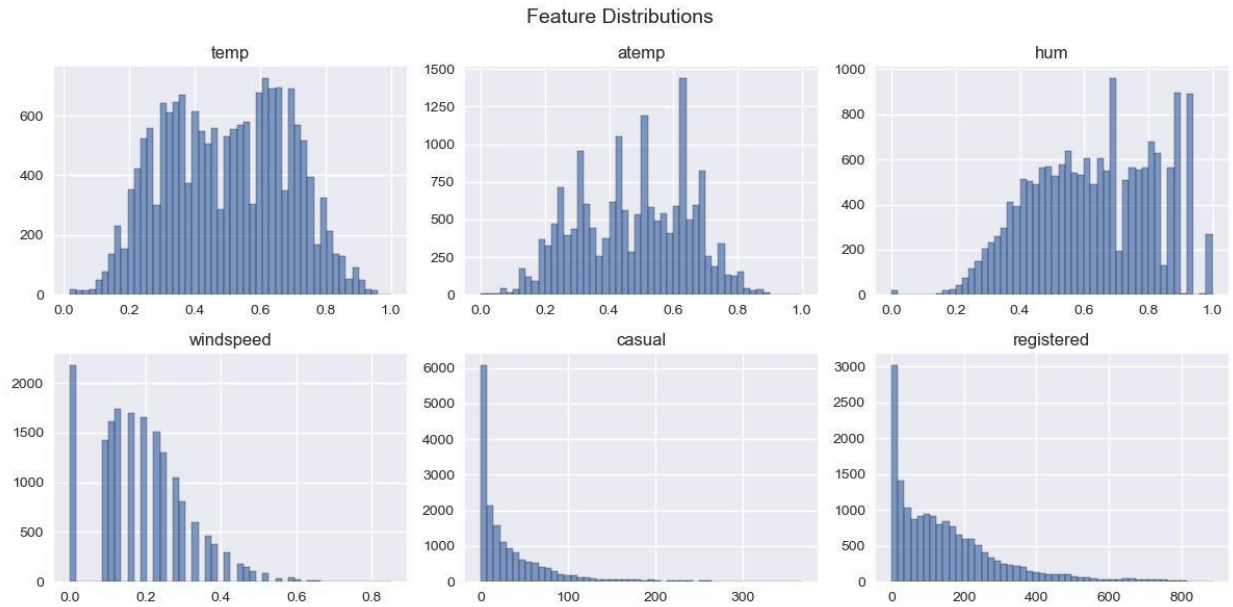


The heatmap above shows that temperature-related variables (`temp`, `atemp`) have a strong positive correlation with the target, indicating that higher temperatures are associated with increased bike rental usage. Conversely, humidity shows a negative correlation with the target, suggesting that there is reduced demand for bike rentals in humid conditions.

The high correlation between `casual`, `registered`, and the target `cnt` confirms the direct relation between the variables, supporting the decision to drop due to data leakage.

3. Exploratory Data Analysis

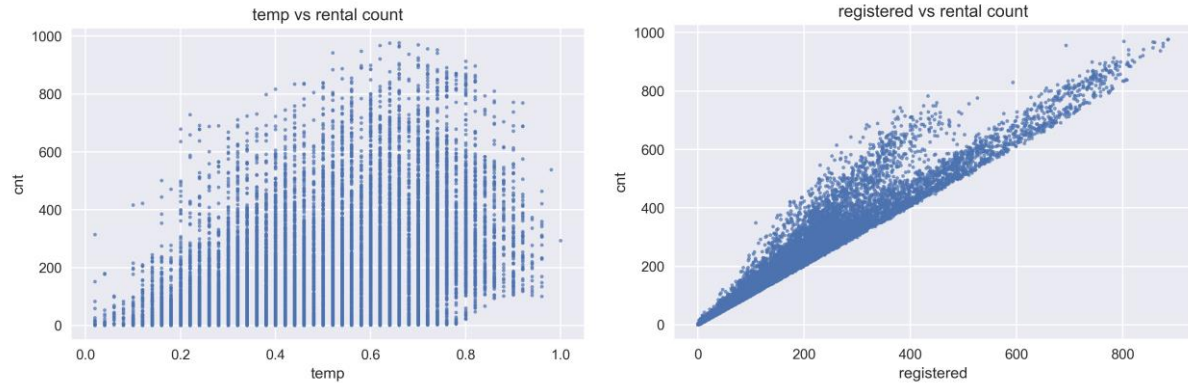
Figure 2. Key Feature Distributions



One of the most important findings in our EDA process is that several continuous features are right-skewed with clear outliers. Looking at the distribution for `casual`, it is heavily right-skewed, with most values below 100 but contains outliers exceeding 300. Similarly, the distribution for `registered` has majority of its values below 200 and some outliers around 800. This behavior is also displayed in the distribution for `windspeed`, it's concentrated around 0.3 but has a tail that reaches ~ 0.7 . These skewed distributions suggest using a log transformation or robust models.

Another key finding is that temporal variables are well distributed but redundant. Both distribution plots for `temp` and `atemp` are roughly symmetric, centered around 0.5-0.7. It's important to note that the correlation between the two variables is extremely high ($r=0.99$). This makes sense because the feature `atemp` contains almost identical data compared to `temp`. This is because `atemp` records the “feels-like” temperature, which is almost always the same as the actual recorded temperature. This contributes to the discovery of a temporal pattern – that rentals are lower in bad weather and vice versa. In terms of next steps, this behavior suggests one of the two variables should be dropped to avoid multicollinearity.

Figure 3. Key Feature Correlations



A major finding with correlation analysis revealed that temperature has a strong positive relationship with bike rentals. Correlation with the target is typically around ~ 0.60 - 0.65 . As temperature increases, so does the number of bike rentals. Since warmer weather significantly increases bike usage, it reveals that temperature is an important (and one of the strongest) predictor for modeling.

Lastly, the scatter plot for `registered` shows an almost linear relationship with the target. Correlation is around ~ 0.97 - 0.99 . In comparison `casual`, its correlation is ~ 0.60 - 0.75 . This behavior reveals registered users dominate in rentals, and that bike demand is typically driven by registered commuters, and not casual riders.

4. Preprocessing

Basic data-quality checks were completed prior to modeling. There were no missing target values in `cnt`, no missing feature values requiring imputation, no duplicate rows, and no impossible negative values in the numeric columns. Therefore, no rows were removed during the initial cleaning.

Figure 4. Preprocessing Decisions

Column	% Affected	Decision	Reasoning
<code>casual</code> , <code>registered</code>	--	Dropped	These columns caused data leakage because their sum equals the target variable, <code>cnt</code> .
<code>instant</code>	--	Dropped	This was only a row identifier and did not contain meaningful predictive information.

<code>dteday</code>	--	Dropped	The raw date was excluded because the model was not treated as a time-series model, and its useful components were already represented through <code>hr</code> , <code>mnth</code> , and <code>yr</code> .
<code>hum</code>	0.14%	Outliers kept	Values were rare but within the expected humidity range, so they were treated as real environmental variation.
<code>windspeed</code>	1.83%	Outliers kept	Extreme <code>windspeed</code> values were rare but plausible; removing them could reduce the model's ability to learn real weather patterns.

After cleaning, encoding, and preprocessing, the final training set contained 13,903 rows × 53 predictor columns. The data was split using an 80/20 train-test split, with 80% of observations assigned to training and 20% assigned to testing. Because this was a regression task rather than a classification task, a standard random train-test split was applied without stratification.

5. Feature Engineering

The following engineered features were created to capture time-of-day behavior, peak commute-time demand, extremity in weather conditions, and interactions between environmental variables that might influence demand.

Figure 5. Summary of Engineered Features

Name	Technique	Rationale
<code>is_dawn</code>	Binned hours 5:00-7:00	Captured demand patterns early in the day
<code>is_morning</code>	Binned hours 8:00-11:00	Captured morning commute demand
<code>is_afternoon</code>	Binned hours 12:00-16:00	Captured midday usage demand
<code>is_evening</code>	Binned hours 17:00-19:00	Captured evening commute demand
<code>rush_hour</code>	Created a binary indicator for 5:00-7:00 PM on working days	Captured predictable evening commute demand, when bike rentals were expected to be high

temp_extreme	Flagged temp values below the 10 th percentile or above the 90 th percentile	Captured unusually low or high temperatures that could affect rental behavior
atemp_extreme	Flagged atemp values below the 10 th percentile or above the 90 th percentile	Captured extreme “feels-like” temperatures that may influence rider comfort
hum_extreme	Flagged hum values below the 10 th percentile or above the 90 th percentile	Captured unusually dry or humid conditions that could affect demand
windspeed_extreme	Flagged windspeed values below the 10 th percentile or above the 90 th percentile	Captured unusually calm or windy conditions that could influence bike usage
temp_diff	Calculated the difference between atemp and temp	Captured the gap between perceived and actual temperature, which could affect rider comfort and rental decisions
temp_windspeed	Created an interaction term: temp × windspeed	Captured the combined effect of temperature and windspeed on rental demand; windy conditions may change how comfortable a particular temperature feels
hum_windspeed	Created an interaction term: hum × windspeed	Captured the combined effect of humidity and windspeed, which may influence rider comfort than either factor individually

Several engineered features outperformed the raw columns they were derived from. The strongest engineered feature was `rush_hour`, which had the highest absolute correlation with `cnt` at $r = 0.4513$. This outperformed `workingday_1` (0.0232) and individual evening-hour indicators such as `hr_17` (0.3185) and `hr_18` (0.2683), indicating that combining working-day status with evening commute hours captured demand better than either component alone. The binned feature `is_evening` also performed strongly, with a correlation of 0.4386, exceeding the individual evening-hour indicators `hr_17` (0.3185), `hr_18` (0.2683), and `hr_19` (0.1386). This suggests that the grouping of daily hours into the time-of-day bins captured the evening demand pattern more effectively than isolated hourly dummies by themselves. The interaction feature `temp_windspeed` also outperformed the raw `windspeed`, with a correlation of 0.3021 compared to 0.0908, indicating that `windspeed` became more informative when combined with temperature.

However, not all engineered features made such improvements upon the raw data: `temp_diff` (0.2578) was less correlated with `cnt` than both `temp` (0.4021) and `atemp` (0.3990), and the extreme-weather flags all had relatively low correlations below 0.04.

Categorical variables were encoded using one-hot encoding with one reference category dropped. This was applied to `season`, `yr`, `mnth`, `hr`, `holiday`, `weekday`, `workingday`, and `weathersit`. This method was chosen because these columns represented discrete categories rather than continuous quantities, and dropping one reference level prevented perfect multicollinearity in the linear models while preserving category information for the tree-based models.

The continuous numerical predictors `temp`, `atemp`, `hum`, `windspeed`, `temp_diff`, `temp_windspeed`, and `hum_windspeed` were scaled using `StandardScaler`. Binary variables were excluded from scaling, including the one-hot encoded categorical features and engineered indicator variables such as `rush_hour`, `is_morning`, `is_afternoon`, `is_evening`, and the extreme-weather flags, as these features were already represented as 0/1 indicators.

After preprocessing, the modeling dataset started with 12 predictor columns after dropping the target `cnt` and removing four unusable or leakage-causing columns: `casual`, `registered`, `instant`, and `dteday`. One-hot encoding increased the number of predictor columns, producing 53 baseline predictors. Feature engineering then added 12 new features, bringing the full engineered feature set to 65 predictors. During modeling, a refined feature set was also tested by dropping 5 redundant or low-importance features, reducing the feature count from 65 to 60 predictors.

6. Modeling & Results

Model selection was approached by comparing a range of methods from simple to complex. Linear Regression was used as an interpretable baseline, while Ridge and Lasso were included to apply regularization and, in the case of Lasso, support feature selection. Tree-based models, including Decision Tree, Random Forest, and Gradient Boosting, were selected to capture nonlinear relationships and interactions that linear models might miss. Model performance was evaluated primarily using test R^2 , with RMSE included for practical interpretability. Cross-validation and train-test comparisons were used to assess generalization and ensure that the performance was not dependent on a single train-test split.

Three versions of the predictor data were used during modeling. The baseline dataset contained the predictor set from the preprocessing step, without engineered features and scaling. Two versions of the full dataset, one scaled and one unscaled, included the new engineered features, including the time-of-day, rush-hour, extreme-condition, and interaction variables. The baseline Linear Regression model was trained on all three versions of data, and the best performer of these was chosen for cross-validation. The scaled version of the full dataset was used on the regularized linear models. For the tree-based models, the unscaled engineered data was used. The baseline dataset contained 53 predictors, and the full engineered dataset contained 65 predictors.

A feature-refinement step was also employed to test the effect of reducing redundancy and simplifying the predictor set. Feature removal was informed by several criteria: whether features were zeroed out by Lasso, whether they were highly correlated with other predictors, and their relative importance in the baseline Random Forest model. This led to the removal of `hr_11`, `weekday_3`, `is_evening`, `atemp`, and `atemp_extreme`, producing a refined dataset with 60 predictors. This refinement had almost no effect on the linear models, but it decreased performance for the tree-based models. This suggested that Random Forest and Gradient Boosting were able to use some of the removed features to capture nonlinear patterns or interactions. For that reason, the full engineered feature set was used for the tree-based models thereafter, while the refined feature set was used for Ridge Regression as the linear model representative for tuning.

All models trained, including baseline, refined-feature, and tuned versions, are given in Figure 6. The reports training and test performance and is sorted in descending order by the primary metric, test R^2 .

Figure 6. Results Table

Model	R^2 Train	R^2 Test	RMSE Train	RMSE Test	CV R^2 Mean	CV R^2 Std. Dev.
Tuned Random Forest (all)	0.9917	0.9401	16.49	44.25	0.9352	0.0060
Tuned Gradient Boosting (all)	0.9702	0.9395	31.31	44.48	0.9362	0.0051
Default Random Forest (all)	0.9843	0.9374	22.74	45.25	0.9326	0.0059
Default Random Forest (ref.)	0.9824	0.9315	24.08	47.33	0.9284	0.0058
Default Gradient Boosting (all)	0.9007	0.8928	57.19	59.18	0.8904	0.0047
Default Gradient Boosting (ref.)	0.8932	0.8842	59.33	61.52	0.8848	0.0035
Default Decision Tree (all)	0.9264	0.8820	49.24	62.11	--	--

Default Decision Tree (ref.)	0.9155	0.8494	52.76	70.17	0.8622	0.0088
Default Lasso (all)	0.7128	0.7152	97.29	96.48	--	--
Default Lasso (ref.)	0.7127	0.7151	97.31	96.50	0.7099	0.0018
Default LR (ref.)	0.7127	0.7151	97.31	96.50	0.7099	0.0018
Default Ridge (ref.)	0.7127	0.7151	97.31	96.50	0.7099	0.0018
Tuned Ridge (ref.)	0.7127	0.7151	97.31	96.50	0.7100	0.0018
Default LR (all/unscaled)	0.7128	0.7150	97.28	96.52	--	--
Default LR (all/scaled)	0.7128	0.7150	97.28	96.52	--	--
Default Ridge (all)	0.7125	0.7149	97.33	96.54	--	--
Default LR (raw/no FE)	0.6853	0.6893	101.83	100.78	--	--

Note: LR = Linear Regression; FE = feature engineering; ref. = refined feature set; all = full engineered feature set.

The results showed a clear separation between the linear and the tree-based models. The linear models were stable but had limited predictive power, while the tree-based models achieved much higher test performance. This suggested that bike rental demand was not represented well by a purely linear relationship and that nonlinear interactions among time, weather, and working-day features were important.

The linear models, including the tuned Ridge Regression, all performed similarly, with test R^2 values around 0.715. Their training and test performance were nearly identical, indicating stability in generalization rather than overfitting. However, their relatively low R^2 compared with the tree-based models suggested underfitting. That is, the linear models were too simple to capture the more complex and nonlinear patterns driving bike demand.

The tuned Random Forest achieved the highest test-set performance, with a test R^2 of 0.9401 and test RMSE of 44.25. This indicated strong predictive ability and showed that the model could capture complexity in the relationships in the data. However, compared with the other models, it had a slightly larger gap between training and test performance, suggesting some degree of overfitting.

Tuned Gradient Boosting performed almost as well as Random Forest, with a test R^2 of 0.9395 and test RMSE of 44.48. Although its test accuracy was slightly lower, it showed a smaller gap between training and test performance. This suggested that Gradient Boosting generalized well and produced stable predictions on unseen data.

Hyperparameter tuning had different effects across the finalist models. Ridge tuning selected an alpha of 10, but its performance remained essentially unchanged, indicating that the main limitation was not in the regularization strength, but the linear model form. Random Forest tuning slightly improved test performance compared with the baseline Random Forest, increasing test R^2 from 0.9374 to 0.9401. Gradient Boosting showed the largest tuning gain, increasing test R^2 from 0.8928 to 0.9395. After tuning, Random Forest and Gradient Boosting performed nearly identically on the test set, creating a tradeoff between slightly higher accuracy for Random Forest and slightly stronger generalization for Gradient Boosting.

7. Model Selection & Interpretation

The tuned Random Forest model was selected as the winning model. It achieved the strongest test-set performance, with a test R^2 0.9401 and a test RMSE of 44.25 bikes. This meant that the model explained approximately 94% of the variation in hourly rental counts on unseen test data, with prediction errors typically around 44 bikes per hour. The best default model was the baseline Random Forest, which achieved a test R^2 of 0.9374; therefore, hyperparameter tuning improved performance by 0.0027 in terms of R^2 points. Cross-validation further supported the reliability of the final model, with a CV mean R^2 of 0.9352 ± 0.0060 , indicating that performance was consistent across folds and not dependent on a single lucky train-test split. The model did show some overfitting, since train R^2 decreased from 0.9917 to 0.9401 on the test set, for a train-set gap of 0.0516. However, the strong cross-validation performance and low CV standard deviation suggested that this overfitting did not prevent the model from generalizing well.

Residual analysis was used as an additional check because the tuned Random Forest and tuned Gradient Boosting models performed almost identically on the main test metrics. For both models, the actual-versus-predicted plots showed that predictions followed the perfect-prediction line closely, indicating that both models captured the overall structure of hourly demand. Both models also showed some widening in residual spread as predicted rental counts increased, suggesting that high-demand periods were harder to predict precisely. However, the Gradient Boosting residual plot showed a stronger concentration of residuals above zero in some lower-demand cases. The Random Forest residuals were more evenly scattered above and below zero, which indicated slightly cleaner error behavior. Because Random Forest also had the best test R^2 and RMSE, it was selected as the final model despite showing a larger train-test gap than Gradient Boosting.

Figure 7. Residual comparison for tuned Random Forest and tuned Gradient Boosting. Both models tracked actual demand closely, but Random Forest showed slightly more balanced residuals around zero, supporting its selection as the final model.

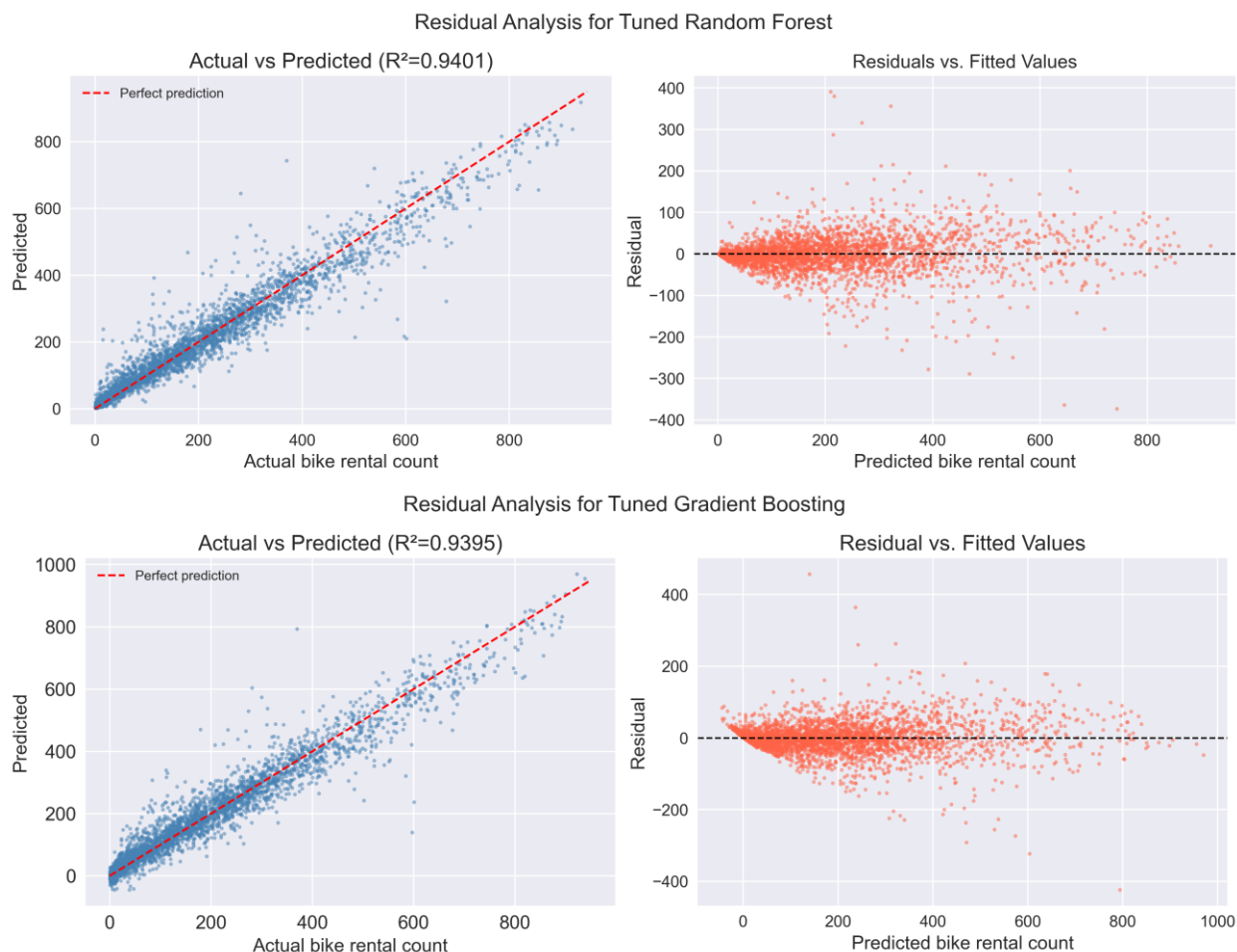


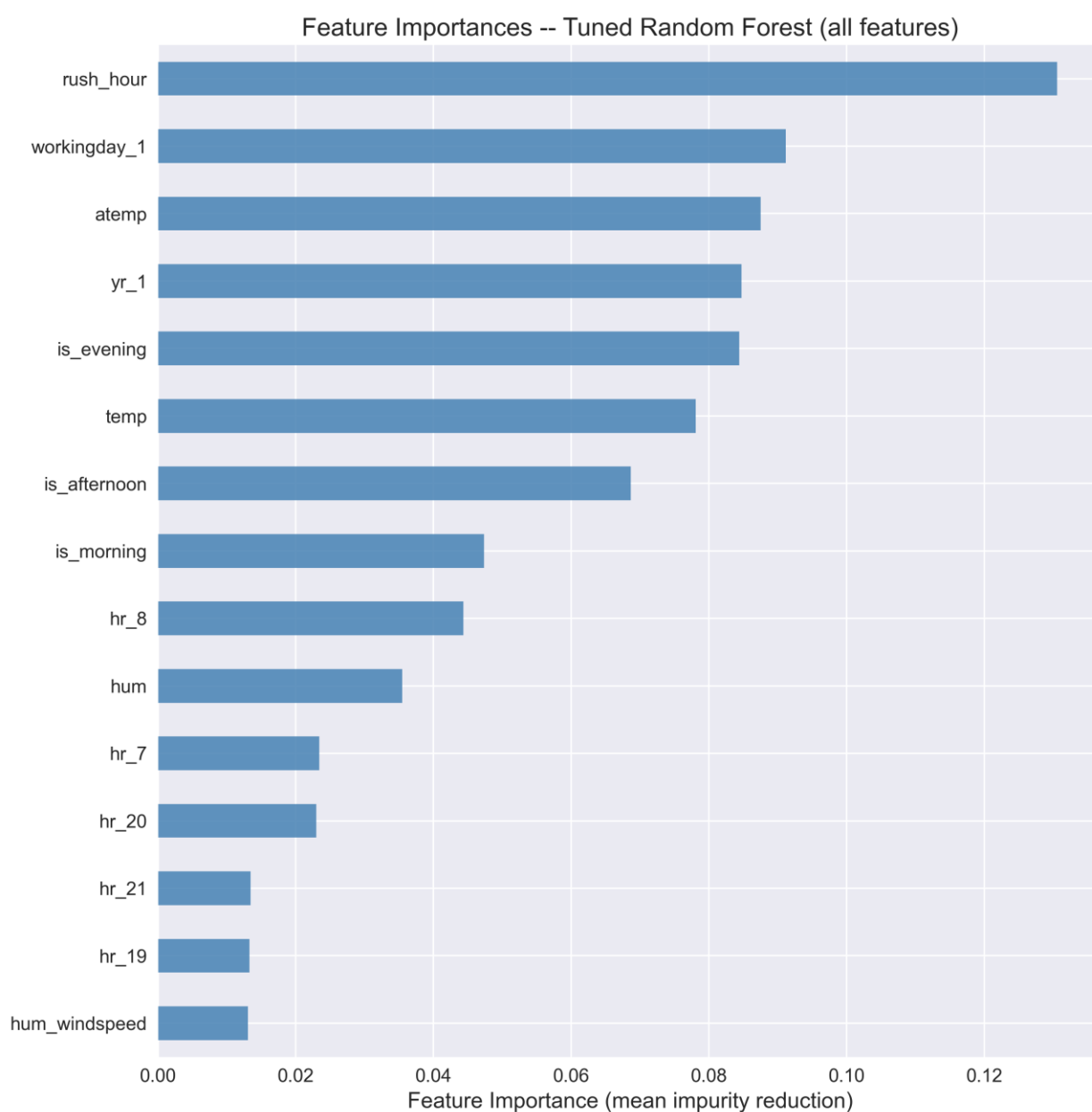
Figure 8 shows the feature importance plot for the tuned Random Forest model. The top three features were `rush_hour`, `workingday_1`, and `atemp`. The most important of these was `rush_hour`, with a value of 0.1306. This engineered binary feature measured whether an observation occurred during the 5-7 PM period on a working day. It was predictive because bike demand was strongly tied to commuting behavior, especially during evening commute times. The business implication for this was that operators should pre-position bikes before evening rush-hour periods to reduce shortages.

The second most important feature was `workingday_1`, with a value of 0.0912. This feature indicated whether the observation occurred on a working day. It was predictive because bike-sharing demand followed different patterns on workdays compared with non-working days, likely reflecting commuter usage as previously noted. The business

implication was that bike distribution strategies should differ between working days and weekends or holidays.

The third most important feature was `atemp`, with a value of 0.0875. This variable represented the “feels-like” temperature, which combined temperature with other weather conditions. It was predictive because riders were more likely to use bikes when conditions felt more comfortable. The business implication was that operators should incorporate weather forecasts into daily planning, increasing bike availability and other staffing and operational readiness when environmental conditions are milder and more comfortable.

Figure 8. Feature Importance Plot for Tuned Random Forest



8. Conclusions & Recommendations

This report examined whether hourly bike-sharing demand could be predicted accurately enough to support better decisions around bike availability and redistribution. The best model was the tuned Random Forest, which achieved a test R^2 of 0.9401 and a test RMSE of 44.25 bikes, indicating strong predictive performance on unseen data.

Based on model results, three practical recommendations for decision makers were identified. First, bikes should be pre-positioned before rush-hour demand peaks. This is informed by the fact that `rush_hour` was the most important feature in the winning Random Forest model. This indicated that commuting behavior was the strongest driver of predicted demand, so bike redistribution should ideally occur before peak commute windows rather than after shortages already start. Second, operators should adjust supply plans based on weather forecasts. This is because temperature was one of the top predictors, showing that more comfortable conditions were strongly associated with increased rentals. This means operators should scale bike count, staff availability, and other resources upward on warmer days and downward when less comfortable temperatures are predicted. Third, operators should maintain a buffer during high-demand periods. This is because the residual plot showed wider error spread at higher predicted rental counts, and the winning model's RMSE was approximately 44 bikes per hour. This means that predictions should be read with caution during times of peak demand, and extra capacity should be on standby to reduce the risk of shortages.

While these recommendations were supported by the model results, they should be interpreted considering several limitations in the data and modeling approach. First, the dataset did not include several factors that likely influence bike demand. Although the model included time and weather variables, it did not include information on the occurrence of special events, bike availability at the station level, pricing, service disruptions, or other real-time factors. As a result, unusual demand spikes may have appeared as unexplained variability in bike count, even though they may have been driven by factors not represented in the data. Second, observations were treated as independent despite the inherently time-based nature of the dataset. The model used decomposed time features such as hour, month, and year,

but did not specifically model temporal dependence, trends, lags in demand, or autocorrelation. Because of this, the model may have been less reliable in situations where recent demand patterns had an influence on future rentals.

One specific next step for this project would be to incorporate station-level data, including pickup/drop-off location, station capacity, and bike availability over time. The current model predicted the total system-wide demand, but it could not identify where bikes were needed. Adding location-level information would allow for the prediction of demand for specific neighborhoods, making results much more useful for actual decisions about bike redistribution.

9. References

Fanaee-T, H. (2013). Bike Sharing [Dataset]. UCI Machine Learning Repository.

<https://doi.org/10.24432/C5W894>

Fanaee-T, H., & Gama, J. (2013). Event labeling combining ensemble detectors and background knowledge. *Progress in Artificial Intelligence*, 2, 113–127.

<https://doi.org/10.1007/s13748-013-0040-3>

Lakshmi, N. P. (n.d.). Bike Sharing Dataset [Data set]. Kaggle.

<https://www.kaggle.com/datasets/lakshmi25npathi/bike-sharing-dataset>